

NexaPay SDK – Documentação de Integração C# (.NET 8)

Bem-vindo à documentação oficial do SDK em C# da **NexaPay**. Esta biblioteca fictícia permite integrar de forma simplificada a nossa gateway de pagamentos robusta e segura na sua aplicação .NET. Suportamos transações por Cartão de Crédito, PIX e Pagamento por Referência Bancária.

1. Configuração Inicial

Para começar a utilizar o SDK, adicione a dependência via NuGet Package Manager ou execute o comando na consola do seu projeto:

```
dotnet add package NexaPay.Sdk.Core
```

Após a instalação, configure o cliente global injetando-o no motor de Inversão de Controlo (IoC) em **Program.cs**:

```
using NexaPay.Sdk;

var builder = WebApplication.CreateBuilder(args);

// Configuração do NexaPayClient
builder.Services.AddNexaPayClient(options =>
{
    options.ApiKey = "nx_live_51Mzk2RkZ2FkZ2FhNDg1N2";
    options.Environment = NexaEnvironment.Production;
    options.TimeoutSeconds = 30;
});
```

2. Autenticação e Segurança

Todos os pedidos efetuados pelo SDK são autenticados através de um token Bearer enviado no cabeçalho HTTP **Authorization**, gerido automaticamente pelo ciclo de vida do cliente. Os dados transmitidos são encriptados via TLS 1.3.

Aviso de Segurança Importante

Nunca exponha a sua **ApiKey** em repositórios públicos ou no lado do cliente (front-end). Utilize sempre variáveis de ambiente seguras ou cofres de chaves (como Azure Key Vault).

3. Criar uma Transação de Pagamento

O exemplo abaixo demonstra como iniciar um fluxo de cobrança utilizando o serviço de transações do SDK.

Exemplo de Código C#

```
using NexaPay.Sdk.Models;
using NexaPay.Sdk.Services;

public class PaymentService
{
    private readonly INexaPayClient _nexaPayClient;

    public PaymentService(INexaPayClient nexaPayClient)
    {
        _nexaPayClient = nexaPayClient;
    }

    public async Task<PaymentResponse> ProcessarPagamentoCartaoAsync()
    {
        var request = new PaymentRequest
        {
            Amount = 150.50m,
            Currency = "EUR",
            PaymentMethod = PaymentMethodType.CreditCard,
            Card = new CardDetails
            {
                HolderName = "MARIA I SILVA",
                CardNumber = "4111222233334444",
                ExpirationMonth = 12,
                ExpirationYear = 2029,
                SecurityCode = "123"
            },
            Metadata = new Dictionary<string, string>
            {
                { "OrderId", "84729" }
            }
        };
    }
}
```

```

        try
        {
            PaymentResponse response = await
            _nexaPayClient.Payments.CreateAsync(request);
            return response;
        }
        catch (NexaPayException ex)
        {
            // Tratamento de erro específico da API
            Console.WriteLine($"Erro ({ex.ErrorCode}): {ex.Message}");
            throw;
        }
    }
}

```

4. Estrutura de Modelos (Data Contracts)

Propriedades de PaymentRequest

Campo	Tipo	Obrigatório	Descrição
Amount	decimal	Sim	O valor total da transação. Deve obedecer à regra $V > 0.00$.
Currency	string	Sim	Código ISO 4217 de três letras (ex: "EUR", "USD", "BRL").
PaymentMethod	Enum	Sim	Tipo de pagamento: CreditCard, Pix, BankReference.
Card	Objeto	Condicional	Obrigatório se o PaymentMethod for CreditCard.

5. Respostas e Tratamento de Erros

A API devolve códigos de estado padrão HTTP. O SDK encapsula erros de validação e de processamento na exceção `NexaPayException`.

Tabela de Erros Comuns

Código HTTP	Código Interno	Causa / Solução
400 Bad Request	INVALID_AMOUNT	O valor especificado é inferior ao mínimo permitido.
401 Unauthorized	AUTH_FAILED	A ApiKey fornecida é inválida ou expirou.
422 Unprocessable	INSUFFICIENT_FUNDS	A instituição bancária recusou a transação por falta de saldo/ limite.

Formato JSON do Erro (Propriedade RawBody)

```
{
  "success": false,
  "error": {
    "code": "INVALID_CARD_EXPIRATION",
    "message": "A data de expiração do cartão está no passado.",
    "details": [
      { "field": "Card.ExpirationYear", "issue": "Ano inválido" }
    ]
  }
}
```

6. Webhooks e Notificações Assíncronas

Para capturar atualizações de estado (ex: pagamento aprovado tardiamente), deve expor um endpoint na sua API para receber os webhooks enviados pela NexaPay.

POST <https://api.oseudominio.com/nexapay-webhook>

```
[HttpPost("nexapay-webhook")]
public async Task<IActionResult> HandleWebhook()
{
    using var reader = new StreamReader(HttpContext.Request.Body);
    var json = await reader.ReadToEndAsync();

    var signature = Request.Headers["Nexa-Signature"];

    if (!_nexaPayClient.Webhooks.ValidateSignature(json, signature, "whsec_segredo"))
    {
        return BadRequest("Assinatura inválida.");
    }

    var webhookEvent = _nexaPayClient.Webhooks.ParseEvent(json);

    if (webhookEvent.Type == WebhookEventType.PaymentSucceeded)
    {
        var paymentId = webhookEvent.Data.Id;
        // Atualizar o estado da encomenda na base de dados
    }

    return Ok();
}
```